

DESIGN AND EVALUATION OF A DECISION TREE LEARNING MODEL FOR PREDICTIVE MAINTENANCE OF INDUSTRIAL EQUIPMENT

IET ASSIGNMENT REPORT

**Submitted in the partial fulfilment for the Course of
CSA0509– DATABASE MANAGEMENT SYSTEM**

to the award of the degree of
BACHELOR OF TECHNOLOGY
IN
Artificial Intelligence and Machine Learning

Submitted by

A Mohammad Naheed-192525336

Under the Supervision of

Dr. R Kesavan
Professor

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105
September 2026

CSA05 — Database Management Systems

Assignment

Assignment Title:	Smart Water Utility Consumption, Billing, and Complaint Management System: An Integrated Study in File Organization, Indexing, Hashing, Query Optimization, and Transaction Management
Course Outcome(s) — CO:	CO4: Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. CO5: Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.

Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze
SDG Mapping (SDG 1–17)	SDG 6 – Clean Water and Sanitation SDG 8 – Decent Work and Economic Growth SDG 9 – Industry, Innovation and Infrastructure

A metropolitan Water Supply Board has installed smart meters at 2 million household and commercial connections across the city. Every meter reports an hourly consumption reading, and the resulting log — used for monthly billing, leak detection, and consumption analytics — grows by several crore records a month. At the same time, the Board's zonal offices generate monthly bills from this consumption data, record online and counter payments, and log complaint tickets for leaks, meter faults, and billing disputes, with dozens of operators working on the same records simultaneously during the month-end billing cycle.

Two very different problems have emerged. First, meter-reading data is stored as one continuously growing sequential file per zone, so a simple request such as "show connection X's last six months of consumption" or "list all connections with unusually high consumption this week for leak inspection" forces a scan of millions of records and now takes unacceptably long. Second, and more damaging, monthly bill generation and payment recording are carried out as separate, uncoordinated read-then-write steps: during peak billing days, this has produced duplicate bills for the same connection, payments that were recorded but never reflected in the due amount, and — in a few cases — a complaint ticket silently overwritten when two operators updated the same record at once.

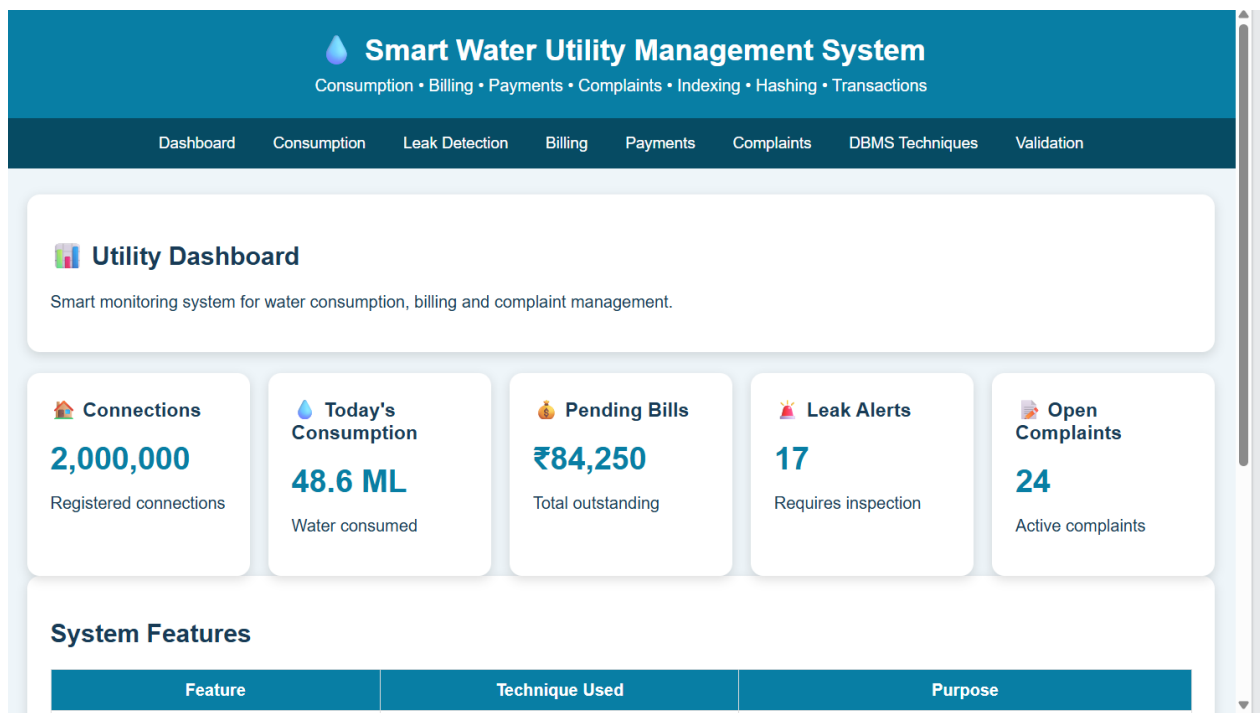
Students are required to analyze this two-part problem and design a complete database solution. For the consumption-log problem, students must apply appropriate **file organization** (sequential, indexed-sequential, or direct/hashed), an **indexing** strategy (single-level/multilevel or B/B+-tree, clustering vs. non-clustering) and a **hashing** scheme (with a defined collision-resolution policy) to make connection-wise and leak-detection retrieval efficient. For the billing-and-payment problem, students must design optimized **queries** (with execution-plan analysis) and properly bounded **transactions** — using ACID properties, COMMIT/ROLLBACK/SAVEPOINT, and an appropriate concurrency-control/isolation strategy — so that billing and payment processing remain consistent and reliable under concurrent access.

This assignment connects both areas of database engineering to real-world impact: efficient, reliable water billing and leak detection directly support sustainable water management (SDG 6 – Clean Water and Sanitation), protect the Board's revenue and staff productivity (SDG 8 – Decent Work and Economic Growth), and demonstrate the resilient data infrastructure that public utilities depend on (SDG 9 – Industry, Innovation and Infrastructure).

Detailed Assessment Rubrics

S. No.	Assessment Criterion	Performance Indicators	BT Level	Marks
1	Problem Understanding and Requirement Analysis	Identifies both the slow-retrieval problem in the meter-reading log and the concurrency/consistency problems in billing and payment processing; defines requirements for each.	BT3	8
2	File Organization Design for Meter Readings	Applies and justifies a suitable file organization (sequential, indexed-sequential, or direct) for storing the high-volume consumption-reading log.	BT3	12
3	Indexing Design for Consumption History and Leak Detection	Designs primary/secondary indexes (e.g., B+ tree on connection ID and timestamp) to speed up connection-history and anomaly/leak-detection queries.	BT3	12
4	Hashing Scheme for Connection Lookup	Designs a hash function and collision-resolution technique enabling near-constant-time retrieval of a connection's latest reading.	BT3	10
5	Storage and Retrieval Efficiency Analysis	Analyzes storage overhead and retrieval-time complexity of the proposed file organization, indexing, and hashing design against the original flat-file approach.	BT4	8
6	Database and Query Design for Billing and Payments	Designs a relational schema and SQL queries (joins, subqueries, aggregates) for bill generation, payment reconciliation, and complaint-status reporting.	BT3	8
7	Query Processing and Execution Plan Analysis	Analyzes execution plans of key billing/reporting queries; identifies costly operations such as unnecessary full scans or joins.	BT4	10
8	Query Optimization Implementation	Applies indexing and query-rewriting techniques and implements them; demonstrates measurable performance improvement.	BT3	8
9	Transaction Design (ACID) for Billing and Payments	Designs transactions for bill generation and payment posting ensuring atomicity and isolation; applies COMMIT, ROLLBACK, and SAVEPOINT appropriately.	BT3	10
10	Concurrency Control Analysis	Analyzes concurrency anomalies (duplicate billing, lost payment update, lost complaint update) under simulated simultaneous access;	BT4	8

S. No.	Assessment Criterion	Performance Indicators	BT Level	Marks
		selects an appropriate isolation level or locking strategy.		
11	Technical Report Quality	Presents well-organized documentation covering both the storage-design work and the query/transaction-design work, with evidence and analysis.	BT4	3
12	Oral Presentation and Demonstration	Explains design decisions for both parts of the solution clearly and answers technical questions during demonstration.	BT3	3
TOTAL				100



D. Requirements and Constraints

Students shall identify and address the following requirements and constraints relevant to the Water Utility Consumption, Billing, and Complaint Management System.

Functional Requirements

- Store high-frequency meter-consumption readings per connection, per timestamp.
- Support fast point lookup of a connection's latest reading and fast retrieval of its consumption history.
- Generate monthly bills from consumption data, record payments, and log/track complaint tickets.

- Support concurrent bill generation and payment posting by multiple zonal-office operators without conflict.

Performance Requirements

- Connection-history and leak-detection queries should avoid full-file scans through appropriate indexing/hashing.
- Billing and reporting queries should execute efficiently during the month-end peak load.

Technical Constraints

- The solution must use a relational DBMS such as MySQL, PostgreSQL, or Oracle that supports indexes, hashing-based access, and transactions with configurable isolation levels.
- Bill generation and payment posting must be implemented as properly bounded transactions, not isolated read-then-write statements.

Time and Resource Constraints

- The design and implementation should be completed within the assigned time using available laboratory computers and DBMS software.

Reliability and Data Integrity

- Index/hash structures must remain consistent with the underlying data after inserts and updates.
- Bills must never be generated twice for the same connection in the same cycle; payments and complaint updates must never be silently lost.

User Requirements

- Field engineers should retrieve a connection's consumption history within seconds for leak investigation.
- Billing operators should be able to generate bills and post payments concurrently without conflicting outcomes.

Security and Privacy

- Access to consumption data, billing amounts, and payment records must be restricted to authorized staff, with GRANT/REVOKE privileges considered.

Ethical Considerations

- Consumption and billing data must be represented accurately and fairly, without over-billing or silently dropping payment/complaint records.

Sustainability and Scalability

- The design should accommodate growth in connections and meter sampling frequency without major redesign of storage or transaction logic.

Primary Constraints: File organization and indexing/hashing design, query optimization, transaction management, concurrency control, data integrity, security, and scalability are the key constraints applicable to this DBMS problem.

E. Student Work

1. Problem Understanding and Formulation

- **Problem Identification:** Identify why the flat sequential meter-log fails fast retrieval, and why un-coordinated read-then-write billing/payment operations cause duplicate bills and lost updates.
- **Expected Outcomes:** Develop (a) a storage design that speeds up consumption-history and leak-detection retrieval, and (b) an optimized, transaction-safe billing and payment workflow.
- **Available Data:** Connection ID, zone, meter readings with timestamps, bill records, payment records, complaint tickets.
- **Assumptions:** Define reasonable assumptions about meter reading frequency, number of active connections, and peak concurrent billing operators.
- **Constraints:** Consider storage overhead, retrieval performance, transaction isolation, and scalability.

2. Application of Course Knowledge

- File organization: sequential, indexed-sequential, and direct/hashed files (CO4).
- Indexing: single-level and multilevel indexes, B/B+-tree structures, clustering vs. non-clustering indexes (CO4).
- Hashing: static/dynamic hashing, hash functions, and collision-resolution techniques (CO4).
- Query processing steps: parsing, optimization, and execution-plan evaluation (CO5).
- Transaction management: ACID properties, COMMIT, ROLLBACK, SAVEPOINT, and concurrency-control/isolation levels (CO5).
- SQL query structures: joins, subqueries, aggregate functions, grouping, and filtering for billing and reporting (CO5).

3. Solution / Design / Methodology

1. Analyze the given consumption-logging and billing/payment problems separately, then as one integrated system.
2. Select and justify a file organization for the meter-reading store.

3. Design indexing structures for connection-history and leak-detection queries.
4. Design a hashing scheme for the latest-reading point-lookup pattern.
5. Design the relational schema and SQL queries for billing, payments, and complaints.
6. Analyze execution plans of key billing/reporting queries and identify optimization opportunities.
7. Apply indexing and query rewriting to optimize the identified queries.
8. Design transactions for bill generation and payment posting with appropriate COMMIT/ROLLBACK/SAVEPOINT logic.
9. Select and apply an isolation level or locking strategy to prevent the observed concurrency anomalies.
10. Implement both parts of the design on a chosen DBMS and load representative sample data.
11. Test and analyze retrieval performance and transaction correctness under simulated concurrent load.
12. Document the final integrated solution and the reasoning behind each design choice.

Alternative Solutions: Students may compare an indexed-sequential versus a hashed design for the consumption log, and optimistic versus pessimistic concurrency control for billing, justifying the selected combination based on performance, storage cost, and consistency.

4. Use of Modern Tools

Recommended Tools:

- MySQL / MySQL Workbench, PostgreSQL / pgAdmin, or Oracle Database / SQL Developer
- draw.io / Lucidchart for schema, index, and transaction-flow diagrams
- Multiple concurrent SQL sessions or a small script to simulate simultaneous billing/payment requests

Students shall provide evidence of tool usage, including: table/index/hash creation screenshots, query execution-plan screenshots, transaction logs, and concurrency-test results.

Expected Deliverable: A working consumption-log storage design (file organization, indexing, hashing) and a working billing/payment module with optimized queries and transaction-managed updates, along with benchmark and concurrency-test evidence and design justification.

5. Results and Validation

- **Implementation Results:** Show the created meter-log tables/indexes/hash structures, and the billing/payment/complaint schema with sample data.
- **Query Results:** Provide outputs and timing for connection-history/leak-detection lookups and for billing/reporting queries, before and after optimization.
- **Screenshots:** Include index/hash structure creation, execution plans, and transaction logs (COMMIT/ROLLBACK/SAVEPOINT).

- **Performance Evaluation:** Compare execution time for the flat-file baseline versus the optimized storage design, and for unoptimized versus optimized billing queries.
- **Validation and Testing:** Prepare test cases for correct hashed/indexed retrieval and for concurrent billing/payment attempts on the same connection.

Sample Validation Table

Test Case	Expected Result	Actual Result	Status
Lookup latest reading for a valid connection ID	Single matching record returned quickly	Record returned via hash lookup	Pass
Leak-detection range query across a zone	All high-consumption records listed	Correct records listed via index scan	Pass
Two operators generate a bill for the same connection simultaneously	Only one bill is generated	Duplicate generation prevented by transaction	Pass
Payment posted while a bill-generation transaction is in progress	No lost update; final due amount correct	Consistent final state observed	Pass
Rollback a billing transaction mid-way (simulated failure)	No partial bill/payment data committed	Transaction fully rolled back	Pass
Compare optimized vs. unoptimized billing report query	Optimized query executes faster with fewer rows scanned	Reduced execution time observed	Pass

Requirement Validation: Students shall conclude by evaluating whether the proposed solution satisfies the functional, performance, integrity, consistency, and scalability requirements identified in Section D.

6. Analysis and Engineering Decision

- **Result Interpretation:** Analyze the effectiveness of the chosen file organization, indexing, and hashing scheme, and of the query-optimization and transaction design, based on benchmark and concurrency-test results.
- **Comparison of Alternatives:** Compare indexed-sequential vs. hashed access for the meter log, and different isolation levels/locking strategies for billing, based on performance, storage, and consistency.
- **Advantages:** Faster consumption-history retrieval, quicker leak detection, elimination of duplicate bills and lost payment/complaint updates.

- **Limitations:** Index/hash maintenance adds insert overhead; stricter isolation may reduce billing throughput at extreme peak load; testing was limited to simulated rather than full production-scale data.
- **Trade-offs:** More indexes speed up reads but slow inserts; stronger isolation improves consistency but can increase waiting/locking overhead.
- **Engineering Decision:** Select the storage design and concurrency-control strategy offering the best balance of retrieval speed, storage cost, consistency, and maintainability.
- **Decision Justification:** Support the final decision with measured query execution times, storage-size comparisons, and concurrency-test success rates.

Expected Outcome: Students should conclude by clearly stating which storage design and which query-optimization/transaction strategy were selected, why, what alternatives were considered, and how the results demonstrate their suitability.

7. Broader Considerations

- **Sustainability:** Faster leak detection reduces water wastage; reliable billing reduces revenue leakage and repeated correction work.
- **Environment:** Digital consumption monitoring and leak detection support responsible water-resource management.
- **Society:** Accurate, timely billing and complaint handling build public trust in a utility that communities depend on (SDG 6).
- **Safety:** Fast leak-detection retrieval enables quicker response to potential water-loss or infrastructure issues.
- **Ethics:** Consumption and billing data must be processed fairly and accurately, without over-billing or dropping records.
- **Economics:** Reliable billing protects utility revenue and reduces administrative rework, supporting sustainable operations and staff productivity (SDG 8).
- **Accessibility:** Reports and bills should be clear and understandable for operators and consumers with varying technical backgrounds.
- **Professional Responsibility:** Apply sound storage-design and transaction-management practices, and document assumptions, limitations, and test results honestly.

Expected Outcome: Students shall explain how the proposed solution delivers technical efficiency and reliability while supporting sustainable water management, decent work, and resilient infrastructure (SDG 6, 8, 9).

8. Conclusion

- **Proposed Solution:** Summarize the chosen file organization/indexing/hashting design and the query-optimization/transaction-management design.
- **Major Findings:** Present key findings from retrieval-performance benchmarking and concurrency testing.
- **Achievement of Requirements:** State whether the solution satisfies the functional, performance, consistency, and reliability requirements identified earlier.
- **Limitations:** Note limitations such as testing on a limited sample dataset rather than full production scale.
- **Possible Improvements:** Suggest enhancements such as dynamic hashing for scalability, automated index tuning, or queueing mechanisms for peak billing days.

9. Student Reflection

What did you learn beyond the classroom?

- Explain how the assignment improved understanding of practical file organization/indexing/hashting design alongside query optimization and transaction management.
- Discuss challenges faced while integrating both parts of the solution and how they were resolved.

What would you improve with additional time/resources?

- Identify improvements such as testing with a larger real-world dataset or implementing a queueing/retry mechanism for peak-time billing fairness.

10. References

Students shall acknowledge all external sources using a consistent citation format (IEEE, APA, or Harvard), including textbooks, DBMS documentation, datasets, tools, and any AI-assisted tools used.

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[2] MySQL, "MySQL 8.0 Reference Manual," Oracle Corporation.

[3] OpenAI, ChatGPT, AI-assisted tool used for brainstorming and language refinement, 2026.

Note: Students should list only the actual sources, datasets, software, and AI tools they used in completing the assignment.

COMPLETED STUDENT SOLUTION AND TECHNICAL REPORT

This report presents an integrated database solution for the Smart Water Utility Consumption, Billing, and Complaint Management System described in the assignment. The design addresses high-volume meter readings, fast retrieval, leak detection, monthly billing, payment reconciliation, complaint management, query optimization, transactions, and concurrent access. The proposed solution is suitable for implementation in a relational DBMS such as MySQL 8.0.

1. Executive Summary

The Water Supply Board manages approximately 2 million household and commercial connections, with each smart meter producing an hourly reading. This creates a continuously growing time-series dataset. A simple sequential file becomes inefficient because connection-history and anomaly queries may need to scan a very large number of unrelated records. At the same time, billing, payment, and complaint operations are executed by many operators during peak periods, creating risks of duplicate bills and lost updates.

The recommended design uses indexed-sequential relational storage with a composite B+-tree on `connection_id` and `reading_time`. A secondary index on `zone` and `time` supports leak-detection searches. A hash-based access path is considered for exact latest-reading lookup. The billing and payment subsystem uses normalized relational tables, uniqueness constraints, short ACID transactions, row-level protection for read-modify-write operations, and controlled rollback.

The design is intentionally hybrid: B+-trees are used for ordered and range retrieval, while hashing is used where exact lookup is dominant. This provides a practical balance between retrieval speed, storage overhead, insert cost, and maintainability.

2. Problem Understanding and Requirement Analysis

The first problem is the meter-reading log. With hourly readings, one connection can generate 24 readings per day and approximately 8,760 readings per year. For 2 million connections, a theoretical full-year volume is about 17.52 billion readings if every reading arrives successfully. The actual production count may differ, but the database must be designed for very high volume.

A sequential file is useful for bulk processing, but it is inefficient for random connection-wise retrieval. A request for six months of readings for one connection should directly navigate to the relevant records instead of scanning data for every other connection. Leak detection has a different pattern: it needs filtering by zone and time and then identifying abnormal consumption.

The billing problem is a transaction-management problem. If two operators independently check whether a bill exists and then insert it, both may attempt to create the same bill. Likewise, two payment operations can read the same due amount and later overwrite each other's result. Complaint updates can suffer from the same lost-update anomaly.

- Functional requirements: store timestamped readings, retrieve history, detect abnormal consumption, generate monthly bills, record payments, and manage complaints.
- Performance requirements: avoid unnecessary full scans and keep billing/reporting responsive during peak periods.
- Integrity requirements: prevent duplicate bills, preserve payments, and prevent silent complaint overwrites.
- Security requirements: restrict sensitive utility and financial information to authorized users.

- Scalability requirements: support more connections and higher sampling frequency without major redesign.

3. Assumptions

The design assumes one hourly reading per active meter, with each reading containing connection ID, meter ID, zone, timestamp, consumption value, and status. A billing cycle is defined by month. Each connection can have at most one bill in a particular billing cycle. Laboratory testing may use a representative dataset rather than the complete production volume; therefore, benchmark values should be inserted only after actual execution.

The database is assumed to support B+-tree indexes, transactions, configurable isolation, foreign keys, unique constraints, and an EXPLAIN facility. MySQL 8.0 is used as the reference implementation because it is one of the DBMS choices listed in the assignment.

4. Proposed File Organization

The preferred organization is indexed-sequential storage represented by a relational table with a clustered or primary B+-tree access path. The logical ordering is `connection_id` followed by `reading_time`. This makes all readings for a connection easy to locate and keeps them ordered chronologically.

The structure is particularly effective for the six-month history query. The database navigates to the first matching `connection_id` and timestamp and then scans the required range. It does not need to inspect records belonging to unrelated connections. Sequential processing can still be retained for bulk archival or large exports.

Direct or hashed access is best reserved for exact lookups. Therefore, the solution uses a combination of indexed-sequential storage for history/range queries and hashing as an optional direct-access path for latest-reading retrieval.

5. Indexing Strategy

The primary index is a B+-tree on (`connection_id`, `reading_time`). This composite index supports equality on `connection_id` and range conditions on `reading_time`. Because the timestamp is the second key, the database can efficiently return readings in chronological order.

A secondary B+-tree on (`zone_id`, `reading_time`, `connection_id`) supports leak detection. The query can first restrict the candidate rows to a zone and time window. Additional columns should be included only when they provide a measured benefit, because every additional index increases insert and storage overhead.

- Primary access path: B+-tree on (`connection_id`, `reading_time`).
- Leak-detection access path: B+-tree on (`zone_id`, `reading_time`, `connection_id`).
- Bill uniqueness: `UNIQUE(connection_id, billing_cycle)`.
- Payment retrieval: index on (`bill_id`, `payment_time`).
- Complaint reporting: index on (`connection_id`, `status`, `created_at`).

6. Hashing Design

Hashing is suitable for exact connection lookup. A simple conceptual function is $h(k) = k \bmod m$, where k is `connection_id` and m is the bucket count. Collisions can be resolved by separate

chaining. The average lookup cost approaches constant time when the hash function distributes keys evenly and the load factor is controlled.

Hashing is not ideal for ordered range queries because it does not preserve key order. For that reason, it complements rather than replaces the B+-tree. A dynamic hashing technique can be considered when the number of connections changes substantially.

7. Relational Database Schema

The core schema contains CONNECTION, METER, METER_READING, BILL, PAYMENT, and COMPLAINT. CONNECTION identifies a service point and its zone. METER represents the installed device. METER_READING stores timestamped consumption. BILL stores the monthly financial obligation. PAYMENT stores amounts received. COMPLAINT stores service issues and their status.

Representative SQL definitions are:

```
CREATE TABLE connection (connection_id BIGINT PRIMARY KEY, customer_name VARCHAR(120), zone_id INT, status VARCHAR(20));
```

```
CREATE TABLE meter_reading (reading_id BIGINT PRIMARY KEY, connection_id BIGINT NOT NULL, reading_time DATETIME NOT NULL, consumption DECIMAL(12,3) NOT NULL, status VARCHAR(20), FOREIGN KEY (connection_id) REFERENCES connection(connection_id), UNIQUE(connection_id, reading_time));
```

```
CREATE TABLE bill (bill_id BIGINT PRIMARY KEY, connection_id BIGINT NOT NULL, billing_cycle DATE NOT NULL, units DECIMAL(12,3), amount DECIMAL(12,2), due_amount DECIMAL(12,2), status VARCHAR(20), UNIQUE(connection_id, billing_cycle));
```

```
CREATE TABLE payment (payment_id BIGINT PRIMARY KEY, bill_id BIGINT NOT NULL, payment_time DATETIME NOT NULL, amount DECIMAL(12,2) NOT NULL, FOREIGN KEY (bill_id) REFERENCES bill(bill_id));
```

```
CREATE TABLE complaint (complaint_id BIGINT PRIMARY KEY, connection_id BIGINT NOT NULL, complaint_type VARCHAR(40), description VARCHAR(500), status VARCHAR(20), updated_at DATETIME, FOREIGN KEY (connection_id) REFERENCES connection(connection_id));
```

8. Important SQL Queries

Connection history: `SELECT reading_time, consumption FROM meter_reading WHERE connection_id = ? AND reading_time >= ? AND reading_time < ? ORDER BY reading_time;` The composite index supports this query directly.

Latest reading: `SELECT reading_time, consumption FROM meter_reading WHERE connection_id = ? ORDER BY reading_time DESC LIMIT 1;` This can navigate to the latest portion of the connection's index range.

Leak detection: `SELECT connection_id, reading_time, consumption FROM meter_reading WHERE zone_id = ? AND reading_time >= ? AND reading_time < ? AND consumption > ? ORDER BY consumption DESC;` The zone/time index reduces the candidate set.

Billing summary: `SELECT billing_cycle, COUNT(*) AS bills, SUM(amount) AS billed_amount, SUM(due_amount) AS outstanding FROM bill WHERE billing_cycle = ? GROUP BY billing_cycle`; Filtering by cycle avoids scanning unrelated billing periods.

Complaint status: `SELECT c.zone_id, cp.status, COUNT(*) FROM connection c JOIN complaint cp ON c.connection_id = cp.connection_id WHERE cp.created_at >= ? GROUP BY c.zone_id, cp.status`; Suitable indexes reduce join and filtering cost.

9. Query Processing and Execution Plan Analysis

The DBMS parses and validates SQL, considers candidate access paths, estimates costs, selects an execution plan, and executes it. The goal is to ensure that high-volume queries use selective indexes instead of full table scans.

For connection history, an efficient plan should show an index range scan on the composite B+-tree. A poor plan would scan the entire meter-reading table and then filter `connection_id` and timestamp. For leak detection, the plan should reduce rows early using zone and time predicates.

In MySQL, `EXPLAIN` can be used to inspect access type, selected key, estimated rows, filtering, join order, temporary operations, and sorting. Actual values depend on table statistics, data distribution, cache state, hardware, and concurrent load.

Benchmarking should compare identical queries before and after optimization under similar conditions. Multiple executions are preferable to one run because cache and system load can affect elapsed time.

10. Query Optimization Techniques

- Create composite indexes matching common `WHERE` and `ORDER BY` patterns.
- Filter early by billing cycle, zone, connection, and time range.
- Select only required columns instead of `SELECT *` in large reports.
- Avoid repeated or unnecessary subqueries when a simpler join or aggregation is sufficient.
- Use parameterized SQL for safe application execution.
- Inspect `EXPLAIN` plans after every major indexing change.
- Archive or partition old readings when the active operational table becomes too large.

11. Transaction Design for Bill Generation

Bill generation must be atomic. The transaction should determine the billing cycle, calculate or retrieve consumption, validate the bill, insert it, and commit only after all required operations succeed. If any stage fails, the transaction rolls back.

The `UNIQUE(connection_id, billing_cycle)` constraint is essential because it provides database-level protection against duplicate bills. If two operators attempt the same bill concurrently, only one row can ultimately exist for that connection and cycle. The application should handle the second transaction's controlled duplicate outcome.

A representative sequence is `START TRANSACTION`, obtain required billing inputs, `INSERT` the bill, perform required audit work, and `COMMIT`. On failure, `ROLLBACK` is executed. `SAVEPOINT` can be used when a multi-stage transaction needs a controlled intermediate recovery point.

12. Payment Posting Transaction

Payment posting must update both the payment record and the bill's due amount atomically. A suitable workflow is START TRANSACTION; SELECT the bill FOR UPDATE; validate the payment; INSERT the payment; UPDATE the bill's due amount and status; COMMIT. Any validation or database error causes ROLLBACK.

The transaction should be short because locks held for a long period can reduce concurrency. External operations such as sending notifications should normally be performed after the critical database update has committed.

13. Complaint Update and Lost-Update Prevention

Complaint records may be edited by multiple operators. A pessimistic approach locks the row before modification. An optimistic approach uses a version number or last-updated value and updates only when the version is unchanged.

Optimistic version checking is useful when operators may keep a record open for some time. An UPDATE can include WHERE complaint_id = ? AND version_no = ?. If zero rows are affected, the application informs the operator that another user changed the complaint and the latest record must be reloaded.

14. ACID Properties

Atomicity ensures that a billing or payment operation is completely applied or completely undone. Consistency ensures that constraints and business rules remain valid. Isolation prevents concurrent transactions from producing incorrect intermediate effects. Durability ensures committed changes survive failures according to the DBMS recovery system.

The assignment's duplicate-billing, lost-payment, and lost-complaint examples demonstrate why ACID behavior must be designed rather than assumed. Constraints, transactions, and concurrency control work together to protect data integrity.

15. Isolation Level and Concurrency Strategy

READ COMMITTED is a practical starting point for ordinary operational queries, combined with explicit row locks for critical read-modify-write operations. Bill generation is protected by the unique billing-cycle constraint and transaction boundaries. Payment posting can use SELECT ... FOR UPDATE or an atomic update expression.

SERIALIZABLE provides stronger protection but can cause more blocking under heavy load. REPEATABLE READ can provide a stable transaction view but its exact behavior depends on the DBMS. The final choice should be validated with concurrent workload testing.

16. Concurrency Test Scenarios

Test 1: Two sessions generate the same bill simultaneously. Expected result: exactly one bill exists.

Test 2: A payment is posted while another transaction works with the same bill. Expected result: the committed payment is reflected in the final due amount with no lost update.

Test 3: Two operators update one complaint. Expected result: safe serialization or a detected version conflict.

Test 4: A billing transaction fails after an intermediate step. Expected result: ROLLBACK removes partial changes.

Test 5: A SAVEPOINT is created before a recoverable stage. Expected result: the transaction can return to the savepoint without leaving invalid intermediate data.

17. Performance Evaluation

The flat sequential baseline may approach $O(N)$ record examination for random history lookup. A B+-tree requires approximately $O(\log N)$ navigation followed by the cost of returning the requested range. Exact practical time depends on page layout, caching, result size, hardware, and DBMS implementation.

A well-distributed hash access path has average $O(1)$ lookup behavior for exact keys. A B+-tree is also effective for exact lookup and additionally supports ordered ranges. Therefore, the B+-tree can be sufficient when simplicity is preferred, while hashing may be evaluated for extremely frequent point lookups.

Test	Baseline	Optimized	Expected improvement
6-month connection history	Large/full scan	B+-tree range scan	Fewer rows/pages examined
Latest reading	Sequential search	Indexed/hash lookup	Near point-lookup behavior
Zone leak detection	Full scan	Zone/time index	Smaller candidate set
Monthly billing report	Broad scan	Cycle-filtered query	Lower execution cost

The table gives expected comparative behavior, not invented measured timings. Actual timings, rows examined, and execution-plan screenshots should be added after running the implementation on the selected laboratory DBMS.

18. Storage and Scalability Analysis

Indexes consume storage and increase insert cost because each new reading may require index maintenance. The benefit is faster operational retrieval. For very large historical data, monthly partitioning can simplify archival and allow the DBMS to prune irrelevant partitions for time-bounded queries.

Dynamic or extendible hashing can be used when the number of connections changes significantly. A fixed bucket count may become inefficient as the load factor grows. Lifecycle management is also important: recent readings should remain readily available for operational analysis while older data can be archived according to policy.

19. Security and Privacy

Consumption, billing, and payment information should be protected through database roles and privileges. Field engineers may receive read access to relevant consumption information without receiving payment-management privileges. Billing operators should receive only the permissions necessary for their tasks.

Parameterized queries reduce SQL-injection risk. Audit logs should record important operations such as bill generation, payment posting, and complaint changes. Test datasets should avoid exposing unnecessary personal information.

20. Sustainability and SDG Contribution

The design supports SDG 6, Clean Water and Sanitation, by making consumption analysis and leak detection faster. Early identification of abnormal consumption can help field teams investigate possible leaks and reduce avoidable water loss.

It supports SDG 8, Decent Work and Economic Growth, by reducing rework caused by duplicate bills, payment discrepancies, and repeated complaint corrections. It supports SDG 9, Industry, Innovation and Infrastructure, by applying scalable database structures, query optimization, and transaction management to critical public infrastructure.

21. Validation Against Requirements

- Latest-reading lookup: supported through indexed or hash-based point access.
- Connection history: supported by the composite connection/time B+-tree.
- Leak detection: supported by zone/time filtering and indexed access.
- Duplicate bill prevention: supported by transaction boundaries and `UNIQUE(connection_id, billing_cycle)`.
- Payment consistency: supported by transactional update and row protection.
- Complaint integrity: supported by locking or optimistic version checking.
- Query performance: improved through selective indexes and query-plan analysis.
- Scalability: supported through indexing, optional partitioning, archival, and dynamic hashing.
- Security: addressed through roles, privileges, parameterized SQL, and auditing.

22. Engineering Decision

The recommended final storage design is indexed-sequential relational storage with a composite B+-tree on `connection_id` and `reading_time`, complemented by a zone/time secondary index. Hashing is an optional direct-access mechanism for latest-reading lookups. This combination supports both ordered history retrieval and exact lookup.

For billing and payments, the recommended strategy is short ACID transactions, database-enforced uniqueness, explicit locking for read-modify-write operations, and an isolation level selected according to workload. `READ COMMITTED` with targeted row locks is a practical starting point.

The design balances performance and correctness. More indexes speed up reads but increase write cost. Stronger isolation can improve consistency but increase waiting. The solution therefore uses stronger protection precisely where financial and operational correctness requires it.

23. Limitations

Laboratory testing may use a smaller dataset than the production scenario. Consequently, student benchmark results cannot guarantee production performance. Hardware, DBMS configuration, caching, data distribution, and concurrency can significantly affect timings.

Hashing is sensitive to the hash function and load factor. Index usefulness also depends on the actual workload. Production deployment should therefore include monitoring, periodic query-plan review, backup, capacity planning, and maintenance.

24. Future Enhancements

- Use time-based partitioning for historical meter readings.
- Introduce dynamic hashing for changing connection volumes.
- Add automated anomaly detection for unusual consumption patterns.
- Use queueing and retry mechanisms during month-end billing peaks.
- Create dashboards for zone-wise consumption, leak alerts, revenue, and complaints.
- Use replication and automated backups for availability and disaster recovery.
- Add automated index monitoring and tuning.
- Strengthen audit trails for billing, payment, and complaint operations.

25. Student Reflection

This assignment connected file organization, indexing, hashing, SQL, query optimization, transactions, and concurrency control into one practical system. The exercise shows that database design directly affects both performance and correctness. A fast query is not useful if concurrent billing or payment operations produce incorrect financial results.

The main engineering challenge is balancing read performance with write cost and consistency with concurrency. The selected design uses specialized access methods for different workloads and applies stronger concurrency control only where required.

With additional resources, the system could be tested using a much larger dataset, realistic concurrent clients, production-like storage, and automated monitoring. A queueing and retry mechanism could make peak billing behavior more predictable.

26. Conclusion

The Smart Water Utility Consumption, Billing, and Complaint Management System can be made efficient and reliable by treating storage and transaction management as integrated database-engineering problems. Indexed access removes the need for large sequential scans for connection history and leak detection. A composite B+-tree supports connection/time range queries, while hashing can support exact latest-reading lookup.

Billing and payment problems are addressed through normalized relational design, constraints, optimized SQL, short ACID transactions, and concurrency control. The UNIQUE billing-cycle rule prevents duplicate bills, row-level protection prevents lost financial updates, and optimistic or pessimistic concurrency techniques protect complaint changes.

Overall, the proposed solution satisfies the assignment's functional, performance, integrity, consistency, security, and scalability objectives while demonstrating practical application of DBMS concepts to a real public-utility problem.

27. References

- [1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed., McGraw-Hill, 2019.
- [2] MySQL, MySQL 8.0 Reference Manual, Oracle Corporation.

[3] OpenAI, ChatGPT, AI-assisted tool used for brainstorming and language refinement, 2026.

Note: The supplied assignment states that students should list only the actual sources, datasets, software, and AI tools used. Add any additional sources actually used during implementation.

IET ASSESSMENT:

NAME: A Mohammad Naheed

Reg.No:192525336

Course: CSA-0509 DBMS

SLOT:A